# Modelling Software-based Systems

## Correctness by Contruction of sequential recursive algorithms using Event-B Refinement

Master Informatique

Dominique Méry
Telecom Nancy,Université de Lorraine

4 novembre 2026

dominique.mery@loria.fr

# Summary


1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

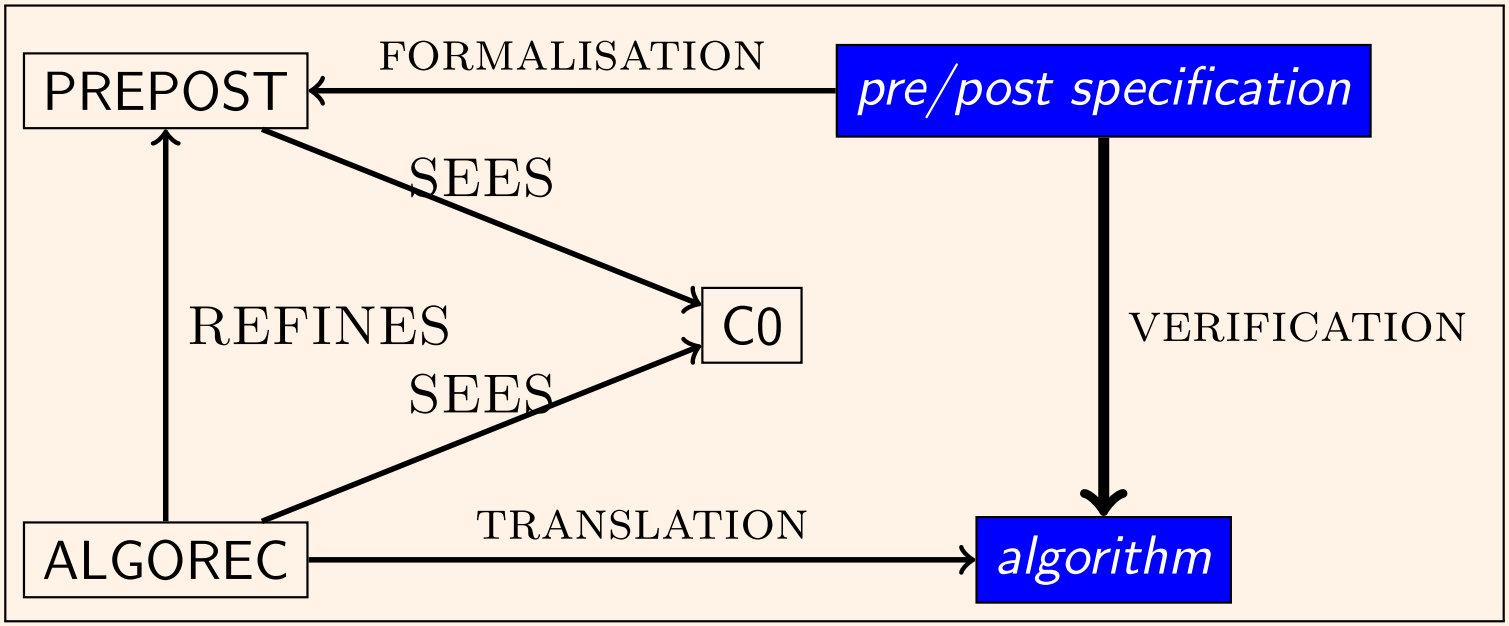
# Current Section Summary


1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

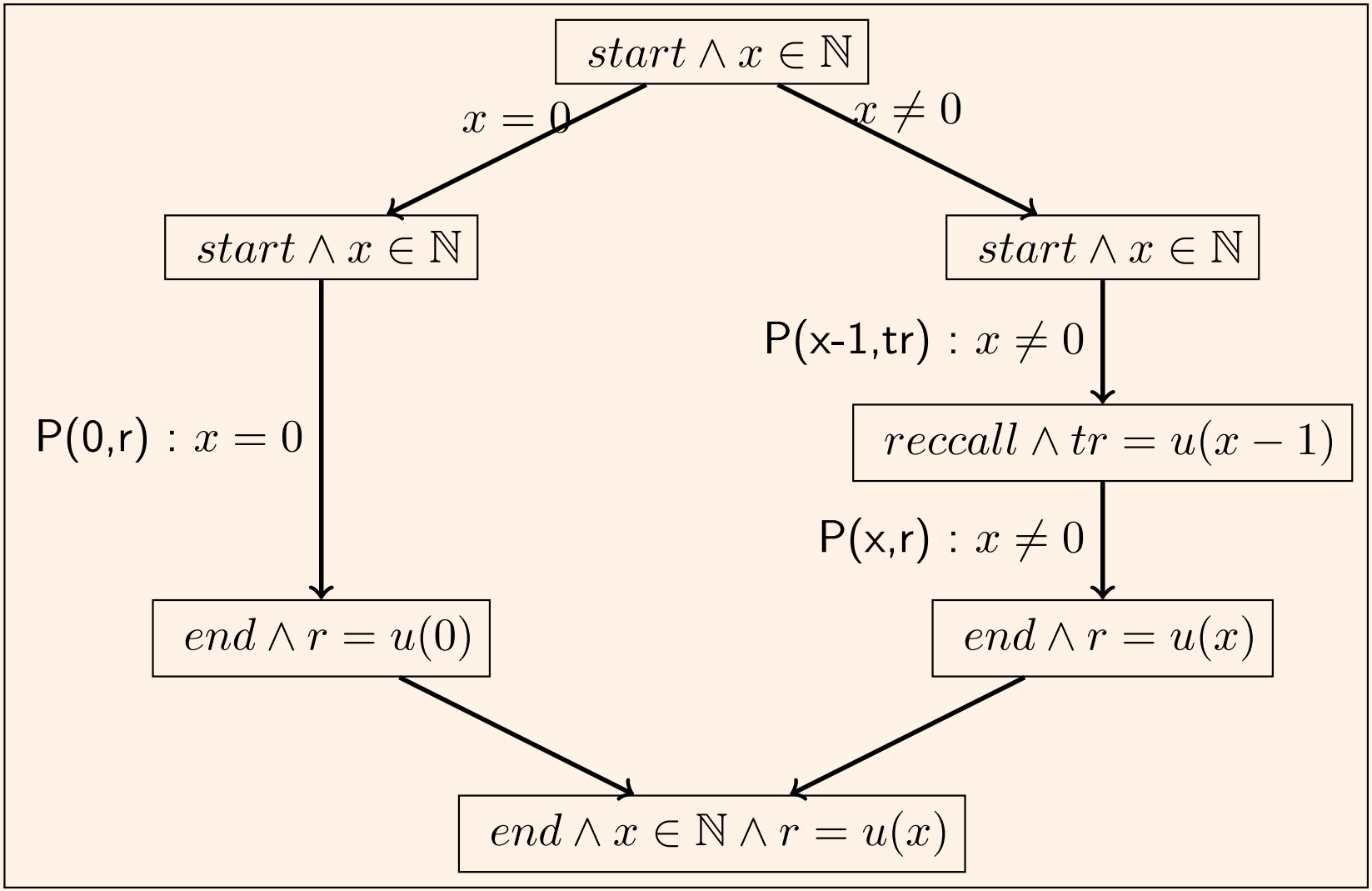
# Current Section Summary


1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

# The recursive pattern



# The refinement diagram



# Ideas for producing algorithms

- **EVENT** rec%PROC(h(x),y)%P(y) is simply simulating the recursive call of the same function.
- The invariant is defined in a simpler way by analysing the inductive structure and a control variable is introduced for structuring the inductive computation.

**EVENT**
e
**WHERE**
$\ell = \ell_1$
$g_{\ell_1,\ell_2}(x)$
**THEN**
$\ell := \ell_2$
$x := f_{\ell_1,\ell_2}(x)$
**END**

**EVENT**
rec%PROC(h(x),y)%P(y)
**ANY** $y$
**WHERE**
$\ell = \ell_1$
$g_{\ell_1,\ell_2}(x, y)$
**THEN**
$\ell := \ell_2$
$x := f_{\ell_1,\ell_2}(x, y)$
**END**

**EVENT**
call%APROC(h(x),y)%P(y)
**ANY** $y$
**WHERE**
$\ell = \ell_1$
$g_{\ell_1,\ell_2}(x, y)$
**THEN**
$\ell := \ell_2$
$x := f_{\ell_1,\ell_2}(x, y)$
**END**

# The computation of the function $x^2$

**VARIABLES**

$r, c, tr$

**INVARIANTS**

$inv1 : r \in \mathbb{N}$

$inv2 : c = end \Rightarrow r = n * n$

$inv3 : c = callrec \Rightarrow n \neq 0$

$inv4 : c = callrec \Rightarrow tr = (n-1) * (n-1)$

$inv5 : c \in C$

$inv6 : tr \in \mathbb{N}$

$inv7 : c = end \Rightarrow r = n * n$

$inv8 : c = end \wedge n \neq 0$
$\Rightarrow tr = (n-1) * (n-1) \wedge r = tr + 2 * (n-1) + 1$

$inv9 : c = callrec \Rightarrow n * n = tr + 2 * (n-1) + 1$

# The computation of the function $x^2$

**EVENT** INITIALISATION
**BEGIN**
$act1 : r := 0$
$act2 : c := start$
$act3 : tr :\in \mathbb{N}$
**END**
**EVENT** square0
**REFINES** square(n ;r)
**WHEN**
$grd1 : c = start$
$grd2 : n = 0$
**THEN**
$act1 : c := end$
$act2 : r := 0$
**END**

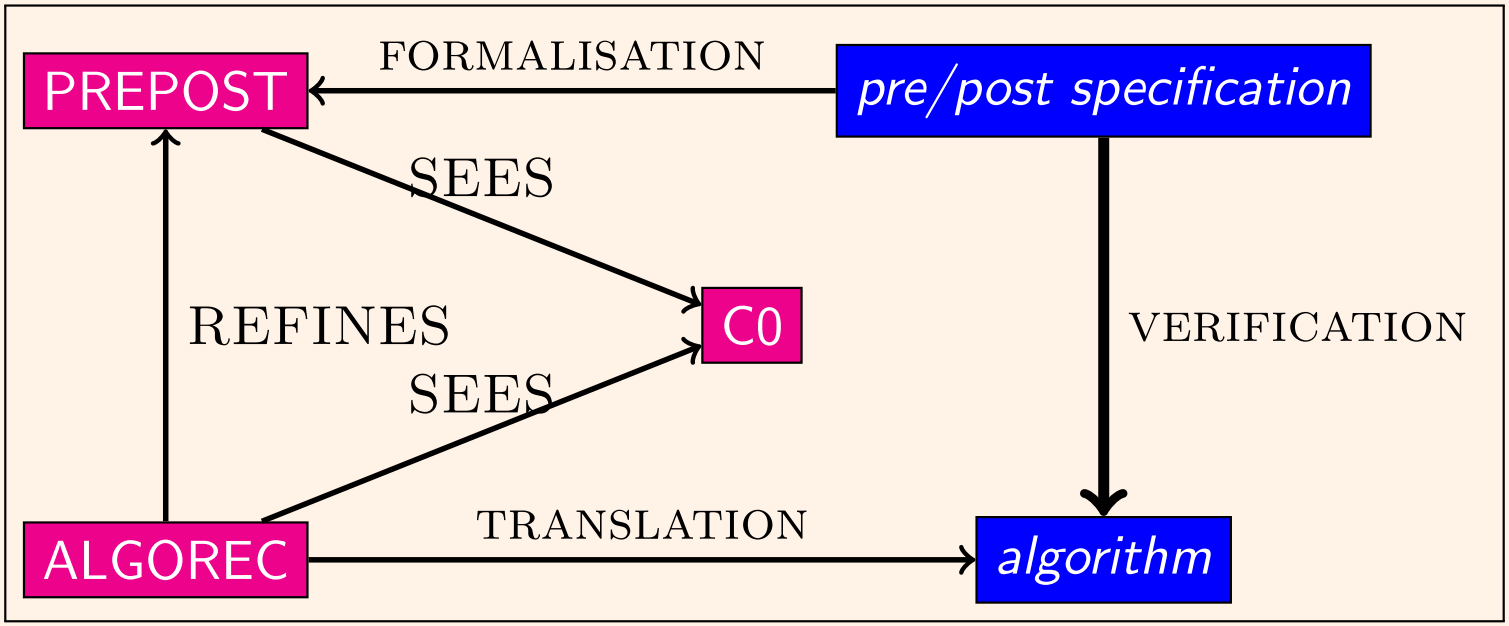
**EVENT** squaren
**REFINES** square(n ;r)
**WHEN**
$grd1 : c = callrec$
**THEN**
$act1 : r := tr + 2 * (n - 1) + 1$
$act2 : c := end$
**END**
**EVENT** rec%square(n-1 ;tr)
**WHEN**
$grd1 : c = start$
$grd2 : n \neq 0$
**THEN**
$act1 : c := callrec$
$act2 : tr := (n - 1) * (n - 1)$
**END**

# Summary for proof obligations

## Summary for proof obligations

| Name | Total | Automatic | Interactive |
|---|---|---|---|
| cae-square | 34 | 32 | 2 |
| square0 | 3 | 2 | 1 |
| specquare | 2 | 2 | 0 |
| square | 29 | 28 | 1 |

# The recursive pattern



# Comments

- Proofs are easier and simpler
- Invariant is simple to find
- Translation is automatic
- Students are not happy with it and tools are not always set for this verification.

# Current Section Summary


1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

# Current Sub Section Summary


1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array

# Current Sub Section Summary


1. CM5
2. The Call-as-Event Paradigm
3. Two examples
   - Computing the square of a number
   - Searching key in an array